

INSTITUTO NACIONAL DE APRENDIZAJE

Núcleo Comercio y Servicios · Subsector Informática y Comunicación

Diseño de algoritmos

Sesión 18 · Operadores, mezcla de tipos y decisiones con if ... else

UA2 · Programación estructurada | Módulo CSTI12010



Persona facilitadora

Giovanni Antonio Coto Calderón

Ruta de la sesión



1. Inicio

Repaso y preguntas
diagnósticas

20 min



2. Desarrollo

Exposición · práctica
guiada · práctica
autónoma

175 min



3. Síntesis

Galería de código con
retroalimentación entre
pares

20 min



4. Próxima sesión

Control de flujo: ciclos y
switch

5 min

Recordemos la sesión anterior



Repaso 1

¿Qué imprime `printf("%.2f\n", 7.0 / 2);`?

Recuerde el formato `%2f`.



Repaso 2

¿Con qué formato se lee un `double` con `scanf`?

No es `%f`.

Antes de empezar, piense y responda



Pregunta 1

¿Cuánto cree que da $7 / 2$ en C si ambos números son enteros?

Piense en trunc de la sesión 9.



Pregunta 2

¿Cómo se escribe en C la condición «nota mayor o igual que 70 Y menor o igual que 100»?

Recuerde Y, O y NO de la sesión 11.

Capacidad 2.1 y saberes de la sesión

Capacidad procedimental 2.1

Emplear los principales elementos semánticos y sintácticos del lenguaje de desarrollo en el paradigma de programación estructurada.



Del Ser: Autonomía

Valor: Honradez



Del Convivir: Comunicación asertiva

Valor: Respeto

Saberes



1. Aritméticos y %



2. Precedencia



3. ++, -- y +=



4. Casting



5. Relacionales y lógicos



6. if ... else y ?:

Operadores aritméticos en C



C tiene **+** **-** ***** **/** y **%** (residuo entero). El resultado depende del **tipo** de los operandos: **int / int da int** (división entera) y si alguno es **double**, el resultado es **double**. C no tiene **^**: la potencia se escribe multiplicando (**r * r**) o con **pow** (sesión 21).



MOD → %

17 % 5 = 2



trunc(a / b) → a / b

Si a y b son int.



^ → **multiplicar**

radio * radio



- unario

-x cambia el signo.

División entera y residuo

El **mismo** $7 / 2$ da **3** o **3.5** según los tipos. En negativos, C **trunca hacia 0** y el residuo toma el signo del dividendo.

division.c

```
#include <stdio.h>
int main(void) {
    printf("%-8s= %d\n", "7 / 2", 7 / 2);
    printf("%-8s= %d\n", "7 % 2", 7 % 2);
    printf("%-8s= %.1f\n", "7.0 / 2", 7.0 / 2);
    printf("%-8s= %.1f\n", "7 / 2.0", 7 / 2.0);
    printf("%-8s= %d\n", "-7 / 2", -7 / 2);
    printf("%-8s= %d\n", "-7 % 2", -7 % 2);
    return 0;
}
```

Salida real (gcc)

```
7 / 2      = 3
7 % 2      = 1
7.0 / 2    = 3.5
7 / 2.0    = 3.5
-7 / 2     = -3
-7 % 2     = -1
```

Nivel	Operadores	Ejemplo	Asociatividad
1 Paréntesis	()	(a + b) * 2 obliga a calcular primero lo que está dentro	—
	Operadores unarios (actúan sobre un solo valor). ++: suma 1 a la variable. contador++ equivale a contador = contador + 1. --: resta 1 a la variable. vidas-- equivale a vidas = vidas - 1.	-x: cambia el signo. Si x vale 5, -x vale -5. Es el menos «unario», distinto de la resta. lok: niega una condición (NO en PSeInt). Si ok vale 1 (verdadero), lok vale 0 (falso), y al revés.	(double) s: casting. Convierte temporalmente el valor de s a double. Si s es el int 245, (double) s / 3 da 81.67 en lugar de 81.
2 Unarios	++ -- - ! (tipo)	-x !ok (double) s	derecha → izq.
3 Multiplicativos	* / % * multiplica, / divide y % da el residuo de una división entera	a * b % c	izq. → derecha
4 Aditivos	+ - + suma y - resta	a + b - c	izq. → derecha
5 Relacionales	< <= > >= < menor que, <= menor o igual, > mayor que, >= mayor o igual.	x >= 70	izq. → derecha
6 Igualdad	== != == pregunta si dos valores son iguales; x == 0 vale 1 si x es cero. != pregunta si son distintos (<> en PSeInt). No hay que confundir == (comparar) con = (asignar).	x == 0	izq. → derecha
7 Y lógico &&	&& a && b es verdadero solo si ambas condiciones son verdaderas (Y en PSeInt). Si a es falsa, ya no evalúa b.	a && b	izq. → derecha
8 O lógico	a b es verdadero si al menos una de las condiciones es verdadera (O en PSeInt). Si a es verdadera, ya no evalúa b.	a b	izq. → derecha
9 Operador condicional ?:	?: a > b ? a : b pregunta si a es mayor que b. Si es verdad, devuelve a; si no, devuelve b. Con eso obtiene el mayor de los dos. Es un if ... else que produce un valor.	a > b ? a : b	derecha → izq.
10 Asignación	= += -= *= /= %= = guarda un valor en una variable.	total += x suma x a lo que ya tenía total (total = total + x).	derecha → izq.

EvaluarPrioridad (sesión 9) en C

Las mismas expresiones dan lo mismo... salvo cuando hay una **división entre enteros**. En PSeInt, g) da **12**; en C, **11**.

prioridad.c

```
#include <stdio.h>

int main(void) {
    // Expresiones de EvaluarPrioridad (sesion 9)
    printf("a) %d\n", 20 - 3 * 4 + 2);
    printf("b) %d\n", 5 + 12 / 4 * 2);
    printf("e) %d\n", 50 % 7 * 2 - 3);
    // 7 / 2 entre enteros da 3
    printf("g) %d\n", 5 + 7 / 2 * 2);
    printf("h) %.1f\n", 5 + 7.0 / 2 * 2);
    return 0;
}
```

Salida real (gcc)

```
a) 10
b) 11
e) -1
g) 11
h) 12.0
```

Asignación compuesta, ++ y --

total += x equivale a **total = total + x**. **++** suma 1 y **--** resta 1. Con **a++** se usa el valor y **después** se incrementa; con **++a**, primero se incrementa.

incremento.c

```
#include <stdio.h>
int main(void) {
    int contador = 0, a = 5, b, c;
    double total = 1000;
    contador++;          // contador = 1
    total += 250;        // total = 1250
    total *= 2;          // total = 2500
    b = a++;             // b = 5 y despues a = 6
    c = ++a;             // a = 7 y despues c = 7
    printf("contador = %d\n", contador);
    printf("total = %.2f\n", total);
    printf("a = %d b = %d c = %d\n", a, b, c);
    return 0;
}
```

Salida real (gcc)

```
contador = 1
total = 2500.00
a = 7 b = 5 c = 7
```

Mezcla de tipos y casting

C **promueve** automáticamente: int con double da double. El **casting (tipo)** convierte a propósito. Debe aplicarse **antes** de dividir.

● ● ● casting.c

```
#include <stdio.h>
int main(void) {
    int suma = 245, cantidad = 3;
    double precio = 9.99;
    printf("%-27s= %d\n", "suma / cantidad",
           suma / cantidad);
    printf("%-27s= %.2f\n", "(double) suma / cantidad",
           (double) suma / cantidad);
    printf("%-27s= %.2f\n", "(double) (suma / cantidad)",
           (double) (suma / cantidad));
    printf("%-27s= %d\n", "(int) precio", (int) precio);
    return 0;
}
```

● ● ● Salida real (gcc)

```
suma / cantidad                = 81
(double) suma / cantidad       = 81.67
(double) (suma / cantidad)     = 81.00
(int) precio                    = 9
```

Relacionales y lógicos: de PSeInt a C

PSeInt	C	Ejemplo en C	Resultado
=	==	<code>nota == 70</code>	1 o 0
<>	!=	<code>anio % 100 != 0</code>	1 o 0
< <= > >=	< <= > >=	<code>retiro <= saldo</code>	1 o 0
Y	&&	<code>edad >= 18 && edad <= 25</code>	1 si ambas
O		<code>nota < 0 nota > 100</code>	1 si alguna
NO	!	<code>!tieneDeudas</code>	invierte

En C no hay tipo lógico básico: una comparación vale **1** (verdadero) o **0** (falso), y en un **if** cualquier valor **distinto de 0** es verdadero.

DescuentosFerreteria (sesión 8) en C

logicos.c

```
#include <stdio.h>

int main(void) {
    // Reglas de DescuentosFerreteria (sesion 8)
    double compra = 80000;
    char pago = 'S';
    int porVolumen = compra >= 100000;
    int porEfectivo = pago == 'S' && compra >= 50000;
    printf("porVolumen  = %d\n", porVolumen);
    printf("porEfectivo = %d\n", porEfectivo);
    printf("aplica      = %d\n", porVolumen || porEfectivo);
    printf("no aplica   = %d\n", !(porVolumen || porEfectivo));
    return 0;
}
```

Salida real (gcc)

```
porVolumen  = 0
porEfectivo = 1
aplica      = 1
no aplica   = 0
```

Aprobacion (sesión 10) en C

Si ... Entonces → `if ( ... ) {`, SiNo → `} else {`, FinSi → `}`. La condición va **entre paréntesis**.

● ● ● Aprobacion.psc (UA1)

```
Algoritmo Aprobacion
    Definir nota Como Real;
    Escribir "Nota final:";
    Leer nota;
    Si nota >= 70 Entonces
        Escribir "Aprobado";
    SiNo
        Escribir "Reprobado";
    FinSi
    Escribir "Fin de la revisión";
FinAlgoritmo
```

● ● ● aprobacion.c (UA2)

```
#include <stdio.h>
int main(void) {
    double nota;
    printf("Nota final: ");
    scanf("%lf", &nota);
    if (nota >= 70) {
        printf("Aprobado\n");
    } else {
        printf("Reprobado\n");
    }
    printf("Fin de la revision\n");
    return 0;
}
```

TarifaMuseo (sesión 11): else if

El **Si** anidado dentro del **SiNo** se escribe en C como una cadena **if ... else if ... else**: se evalúa en orden y se ejecuta **solo el primer** bloque verdadero.

TarifaMuseo.psc (UA1)

```
Algoritmo TarifaMuseo
  Si edad < 12 Entonces
    tarifa <- 1000;
    Escribir "Categoría: niño";
  SiNo
    Si edad < 65 Entonces
      tarifa <- 3000;
      Escribir "Categoría: adulto";
    SiNo
      tarifa <- 1500;
      Escribir "Categoría: adulto mayor";
    FinSi
  FinSi
  Escribir "Tarifa: ", tarifa;
FinAlgoritmo
```

tarifa.c (fragmento)

```
if (edad < 12) {
    tarifa = 1000;
    printf("Categoría: nino\n");
} else if (edad < 65) {
    tarifa = 3000;
    printf("Categoría: adulto\n");
} else {
    tarifa = 1500;
    printf("Categoría: adulto mayor\n");
}
printf("Tarifa: %.2f\n", tarifa);
```

Operador condicional ?:

condición ? valor1 : valor2 devuelve valor1 si la condición es verdadera y valor2 si es falsa. Sirve para **elegir un valor**, no para ejecutar bloques.

condicional.c

```
#include <stdio.h>

int main(void) {
    int numero = 7;
    double a = 12.5, b = 30, mayor;

    // condicion ? valor_si_verdadera : valor_si_falsa
    printf("%d es %s\n", numero,
           numero % 2 == 0 ? "par" : "impar");
    mayor = a > b ? a : b;      // MayorDeDos (sesion 10)
    printf("El mayor es %.2f\n", mayor);
    return 0;
}
```

Salida real (gcc)

```
7 es impar
El mayor es 30.00
```

Tres errores que compilan

gcc genera el programa, pero el resultado es incorrecto. Con **-Wall -Wextra** aparecen estas **advertencias reales**:



Incorrecto

● ● ● Advertencias de gcc

```
if (nota = 70)
// suggest parentheses around
//   assignment used as truth value
if (18 <= edad <= 25)
// comparisons like 'X<=Y<=Z' do not
//   have their mathematical meaning
if (retiro <= saldo);
// suggest braces around empty body
```



Correcto

● ● ● Corrección

```
if (nota == 70)
// == compara; = asigna

if (edad >= 18 && edad <= 25)
// dos comparaciones unidas con &&

if (retiro <= saldo) {
// sin ; antes de la llave
```

Decisiones claras en C



Llaves siempre

Aunque el bloque tenga una línea.



== para comparar

= solo para asignar.



Casting antes de dividir

(double) suma / n



Paréntesis

Aclaran la precedencia.



Orden en else if

De lo más restrictivo a lo general.



Probar los límites

69.9, 70 y 70.1.

Paso a paso: decisiones de la UA1 en C

A

DesgloseBilletes →
billetes.c

B

PromedioNotas →
promedio.c

C

Aprobacion →
aprobacion.c

D

CajeroAutomatico →
cajero.c

E

TarifaMuseo → tarifa.c

F

Errores que compilan

Siga cada paso junto con la persona facilitadora y compruebe el resultado antes de avanzar.

Ahora usted: demuestre lo aprendido



Ejercicio 1 · Descuento

DescuentoPorMonto → descuento.c



Ejercicio 2 · Crédito

AprobacionCredito → credito.c



Ejercicio 3 · Año bisiesto

AnioBisiesto → bisiesto.c



Ejercicio 4 · Validar nota

ValidarNota → nota.c

Documento: S18_Practica_autonoma_Estudiantes.docx · Entrega: carpeta UA2/S18/ del portafolio.

¿Qué aprendimos hoy?

1

¿Cuánto da $7 / 2$ entre enteros y cómo se obtiene 3.5?

2

¿Cómo se escribe en C $\text{anio} \bmod 4 = 0$ Y $\text{anio} \bmod 100 \neq 0$?

3

¿Qué diferencia hay entre `a++` y `++a`?

4

¿Por qué `if (nota == 70)` siempre es verdadero?



Galería de código: `credito.c`

Cada persona deja `credito.c` abierto en su pantalla. En parejas rotativas (3 rondas de 4 min), revise el programa de otra persona y deje una nota con:

- Una fortaleza concreta del código.
- Una mejora: llaves, `==` / `=`, `&&`, comentarios o nombres.
- Un dato de prueba que podría fallar (por ejemplo, 's' en minúscula).

Sesión 19: Control de flujo: ciclos y switch



while y do ... while

Repetir con condición



for

Contar repeticiones



switch

Selección múltiple (Según)

Para pensar: ¿cómo haría que nota.c vuelva a pedir la nota hasta que sea válida, como el Repetir de la sesión 12?